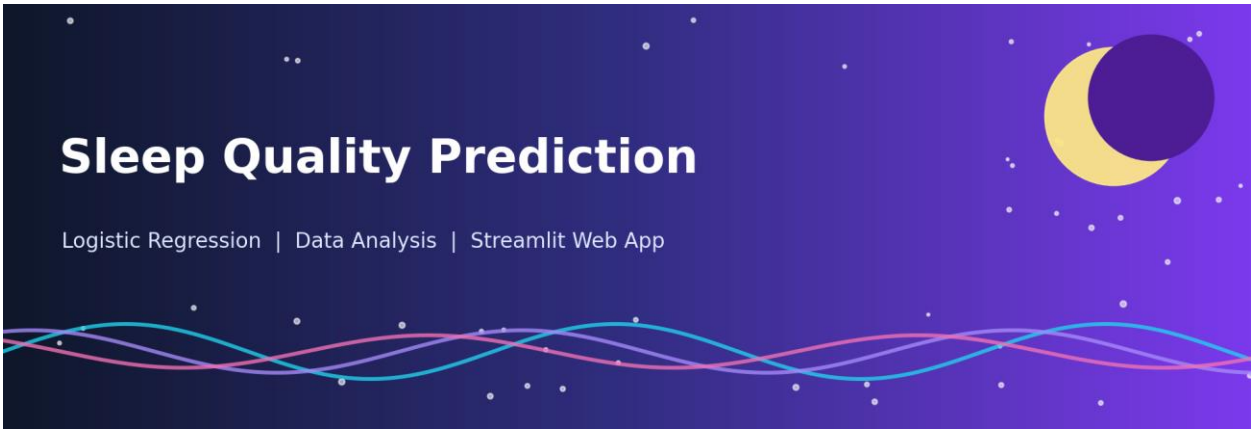
SLEEP QUALITY PREDICTION USING MACHINE LEARNING

NAME: RAHUL RAJ

DESIGNATION: AI/ML INTERN

COMPANY NAME: APANA TIME TECH SOLUTIONS

DATE: [6TH OCTOBER] TO [26TH OCTOBER]



Project Report

Table of Contents:

TOPICS	PAGE
1. Abstract	3
2. Introduction	3
3. Problem Statement	3
4. Objectives	3
5. Scope of the Project	4
6. System Requirements	4
7. Dataset / CSV File	4
8. Data Cleaning and Preprocessing	5
9. Exploratory Data Analysis (EDA)	6
10. Encoding and Train-Test Split	7
11. Logistic Regression Model	7
12. Model Evaluation	8
13. Feature Importance and Interpretation	8
14. Overfitting and Underfitting Analysis	9
15. System Architecture	9
16. Streamlit User Interface	10
17. Personalised Advice Engine	11
18. Complete System Workflow	11
19. Testing and Validation	11
20. Results and Discussion	12
21. Advantages	12
22. Limitations	12
23. Future Scope	13
24. Conclusion	13
25. References	13

1. Abstract

The quality of sleep strongly influences a person's health, concentration and productivity, and it is shaped by everyday lifestyle habits. **Sleep Quality Predictor** is a machine learning project that predicts whether a person's sleep quality is **Good** or **Poor** from five inputs: age, gender, daily screen time, daily caffeine intake and daily exercise hours. The model is a **Logistic Regression** classifier built with scikit-learn, and the project covers the complete machine learning workflow from data cleaning to a working web application.

The dataset is a custom CSV file of **445 records** that deliberately contains **missing values** and **duplicate rows**, so that real pre-processing steps could be practised. After removing 25 duplicate rows, filling the missing values (median for numeric columns, mode for gender) and dropping the identifier column, 420 clean records remained. Exploratory data analysis with histograms, bar charts, boxplots and a correlation heatmap showed that higher screen time and caffeine intake are associated with poor sleep, while more exercise is associated with good sleep.

The model was trained on 80% of the data and tested on the remaining 20%. It reached a test accuracy of **82.1%**, a five-fold cross-validated accuracy of **74.5%** and a ROC-AUC of **0.88**. Training and cross-validated accuracy are almost identical, so the model is not overfitting. Finally, the trained model was deployed in an interactive **Streamlit** application that shows a sleep score gauge, the factors affecting the result, a what-if simulator and personalised advice for better sleep.

2. Introduction

Sleep is one of the most important factors in physical health, mental wellbeing and academic or professional performance. Yet many people sleep poorly because of habits that are easy to measure and change, such as long hours of screen use, frequent caffeine consumption and a lack of physical activity. A data-driven tool that links these habits to sleep quality can help people understand how their routine affects their rest.

This project applies a standard supervised machine learning workflow to that question. The task is a **binary classification** problem: given a person's daily habits, predict one of two classes, Good or Poor sleep quality. Logistic Regression was chosen because it is simple, fast, easy to explain, and gives a probability for each prediction as well as a class label.

The original project brief asked for a small CSV file of 445 rows with three habit columns. This implementation extends that brief: the dataset has 445 rows and also includes an identifier, age and gender, and it intentionally contains nulls and duplicates. The notebook performs cleaning, EDA, encoding, training and evaluation, and the saved model is then used by a Streamlit web application that adds personalised recommendations.

3. Problem Statement

People often do not know which of their daily habits harm their sleep the most. Reading general health articles gives generic advice, but it does not show how a particular combination of screen time, caffeine and exercise is likely to affect a particular person. A predictive model can estimate the chance of good sleep for any combination of habits and show which habits push the result in which direction.

Real-world data is also rarely clean. Records may contain missing entries and repeated rows, and categorical fields must be converted into numbers before a model can use them. A second part of the problem is therefore to build a reliable preprocessing pipeline. Finally, a prediction on its own is not very useful to an ordinary user, so the system should turn the prediction into clear, practical advice through a simple and attractive interface.

4. Objectives

The main objective is to build and evaluate a Logistic Regression model that predicts sleep quality (Good or Poor) from lifestyle habits, and to present it through a user-friendly web application. The specific objectives are:

- To create a custom CSV dataset of more than 400 records with the columns id, gender, age, screen time, caffeine intake, exercise hours and sleep quality, including null values and duplicate rows.

- To clean the data with pandas by removing duplicates, handling missing values and dropping non-informative columns.
- To perform exploratory data analysis using histograms, bar charts, boxplots and a correlation heatmap.
- To encode the categorical variables and split the data into training and testing sets.
- To train a Logistic Regression model in a scikit-learn pipeline with feature scaling.
- To evaluate the model using accuracy, a confusion matrix, a classification report, ROC-AUC and cross-validation, and to check for overfitting or underfitting.
- To develop a Streamlit application that predicts sleep quality for a new user and recommends personalised advice based on the prediction.

5. Scope of the Project

The scope of the project covers the complete machine learning pipeline: dataset creation, data cleaning, exploratory analysis, encoding, model training, evaluation, model saving and deployment through a web interface. The application accepts five inputs, predicts Good or Poor sleep quality with a probability, explains which habits influenced the result and suggests practical improvements.

The project is a learning prototype. The dataset is **synthetic**, so the model demonstrates the machine learning workflow rather than real sleep science. The application is **not a medical tool** and does not diagnose sleep disorders. For a production system, real data from sleep studies or wearable devices, more features (such as sleep duration, stress and diet), user authentication and clinical validation would be required.

6. System Requirements

The project runs on a standard computer and needs no GPU. Python is the main language, and the libraries below are installed with one command:

```
pip install streamlit pandas numpy scikit-learn matplotlib seaborn joblib
```

Category	Requirement	Purpose
Language	Python 3	Application and notebook development
Data handling	pandas, NumPy	Loading, cleaning and numerical operations
Machine learning	scikit-learn	Scaling, Logistic Regression, pipeline and metrics
Visualisation	Matplotlib, Seaborn	Histograms, bar charts, heatmap, confusion matrix, ROC
Model storage	joblib	Saving and loading the trained model (model.pkl)
Web interface	Streamlit	Interactive prediction and advice application
Environment	Jupyter Notebook / VS Code	Running the notebook and editing the code

7. Dataset / CSV File

The dataset is stored in **sleep_quality_data.csv**. It was generated with a Python script (generate_dataset.py) using random distributions: age from a clipped normal distribution, screen time from a normal distribution, caffeine intake from a Poisson distribution and exercise hours from a gamma distribution. The Sleep Quality label was then created from a hidden logistic rule in which more screen time and caffeine reduce the chance of good sleep, more exercise increases it, and random noise is added so that the data is realistic and not perfectly separable. Missing values and duplicate rows were injected on purpose for pre-processing practice.

Column	Type	Description
id	Integer	Unique record number (dropped before modelling)
gender	Categorical	Male, Female or Other

Column	Type	Description
age	Numeric	Age in years (18 to 60)
screen_time	Numeric	Daily screen time in hours (0.5 to 12)
caffeine_intake	Numeric	Cups of caffeinated drinks per day (0 to 8)
exercise_hours	Numeric	Hours of exercise per day (0 to 3)
sleep_quality	Categorical (target)	Good or Poor

Sample of the raw data (NaN marks a missing value):

id	gender	age	screen_time	caffeine_intake	exercise_hours	sleep_quality
230	Male	18.0	11.6	2.0	1.25	Poor
28	Male	34.0	4.3	3.0	0.57	Poor
245	Female	NaN	9.1	2.0	0.33	Poor
158	Female	NaN	1.7	3.0	0.79	Poor
145	Male	36.0	7.8	4.0	0.63	Poor

8. Data Cleaning and Preprocessing

Before cleaning, the dataset had 445 rows, 25 exact duplicate rows and 64 missing values. The cleaning steps were applied in this order:

Column	Missing values (445 rows)	Missing values (after removing duplicates, 420 rows)	Treatment
gender	8	8	Filled with mode (Male)
age	14	12	Median (29)
screen_time	16	15	Median (5.6 h)
caffeine_intake	12	12	Median (2 cups)
exercise_hours	14	14	Median (0.89 h)
id, sleep_quality	0	0	No nulls; id dropped

- **Remove duplicates:** drop_duplicates() removed 25 rows (445 to 420).
- **Drop id:** the identifier carries no predictive information.
- **Fill nulls:** the median is used for numeric columns because it is not affected by outliers, and the mode is used for gender. The alternative, dropna(), would have discarded about 14% of the rows. The target column has no nulls; rows with a missing label would be dropped.
- **Fix types:** age and caffeine_intake were converted back to integers after filling.

```
df = df.drop_duplicates().reset_index(drop=True)
df = df.drop(columns=["id"])
for col in ["age", "screen_time", "caffeine_intake", "exercise_hours"]:
    df[col] = df[col].fillna(df[col].median())
df["gender"] = df["gender"].fillna(df["gender"].mode()[0])
```

After cleaning, the dataset has 420 rows, no missing values and no duplicates.

9. Exploratory Data Analysis (EDA)

Exploratory data analysis was carried out on the cleaned data. The target is fairly balanced: **222 Poor (52.9%)** and **198 Good (47.1%)** records, so accuracy is a fair metric. The sample contains 209 male, 199 female and 12 other respondents.

Feature	Mean	Std. dev.	Min	Median	Max
age	29.8	8.17	18	29	53
screen_time	5.55	2.36	0.5	5.6	12
caffeine_intake	2.59	1.53	0	2	8
exercise_hours	1.01	0.66	0.02	0.89	3

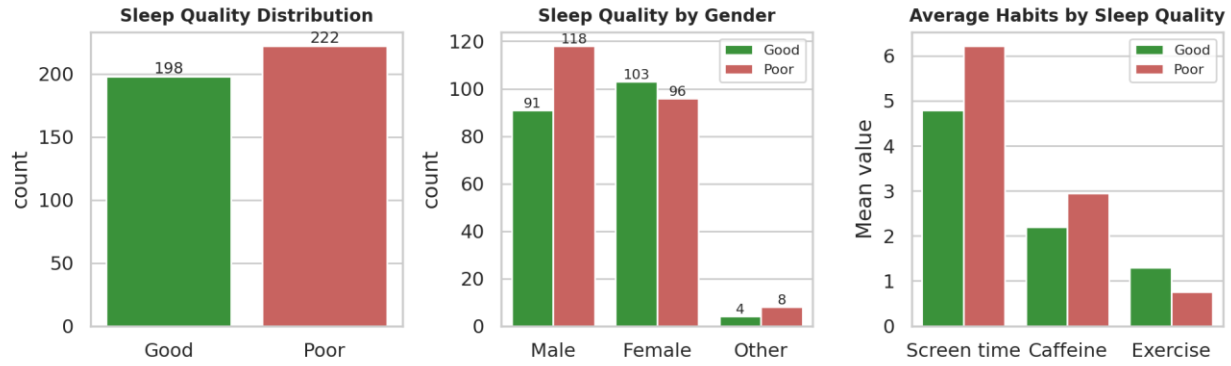


Figure 1: Target distribution, sleep quality by gender and average habits by sleep quality

People with good sleep spend less time on screens (mean 4.79 h against 6.22 h), drink fewer cups of caffeine (2.2 against 2.94) and exercise more (1.3 h against 0.76 h). Average age is almost identical in the two groups.

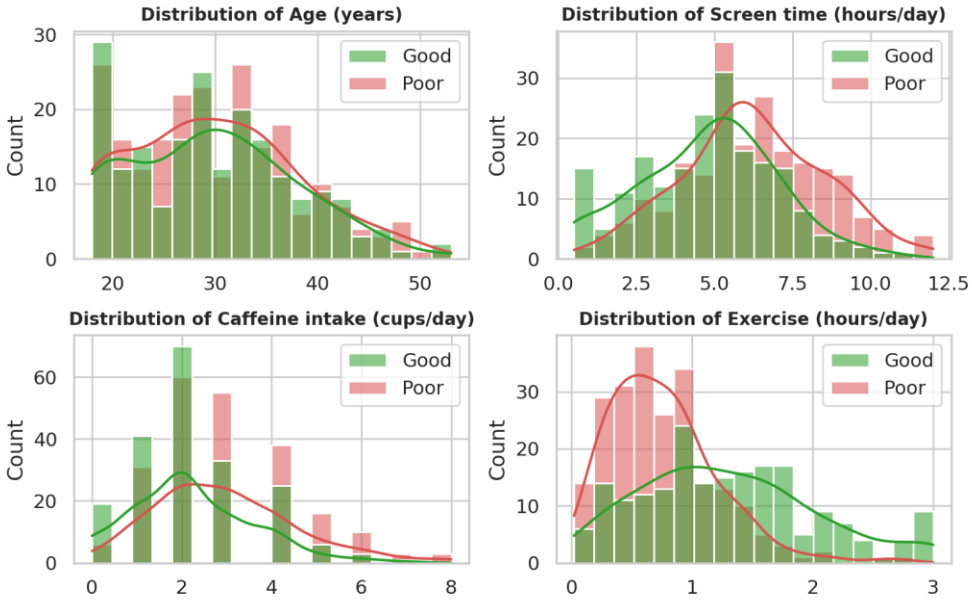


Figure 2: Histograms of the numeric features, split by sleep quality

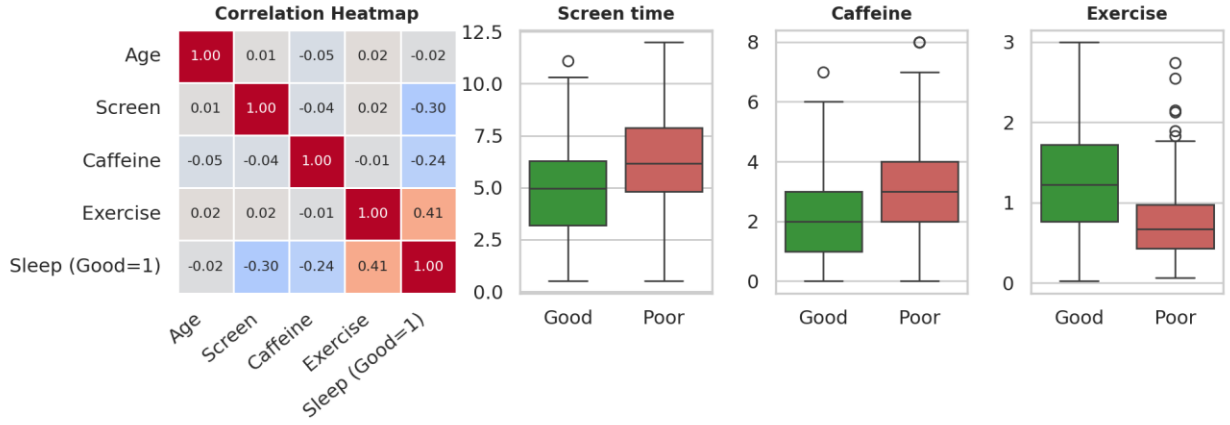


Figure 3: Correlation heatmap and boxplots of the main habits

The heatmap confirms the pattern. Correlation with good sleep is **+0.41** for exercise, **-0.30** for screen time, **-0.24** for caffeine and about **-0.01** for age, so age has almost no effect. The boxplots show that the Poor group has higher screen time and caffeine and lower exercise.

10. Encoding and Train-Test Split

Machine learning models need numeric input, so the categorical columns were encoded. The target **sleep_quality** was mapped to **Good = 1** and **Poor = 0**. The feature **gender** was one-hot encoded with drop_first=True, which creates gender_Male and gender_Other and uses Female as the reference group. The final feature set has six columns: age, screen_time, caffeine_intake, exercise_hours, gender_Male, gender_Other.

The data was split into 80% training and 20% testing sets with train_test_split(test_size=0.2, random_state=42, stratify=y). Stratification keeps the class ratio equal in both sets. The training set has **336** rows (47.0% Good) and the test set has **84** rows (47.6% Good).

11. Logistic Regression Model

Logistic Regression models the probability of the positive class (Good sleep) with the sigmoid function applied to a weighted sum of the features:

$$P(Good) = 1 / (1 + e^{-(b0 + b1x1 + \ldots + b6x6)})$$

If P(Good) is at least 0.5, the person is classified as Good, otherwise Poor. The weights b are learned from the training data by maximising the likelihood. Because the features are on different scales (age in tens, screen time in hours), a StandardScaler is applied first. Both steps are placed in a scikit-learn Pipeline, so the scaler is fitted on the training data only, which prevents data leakage, and the same scaling is applied automatically when the saved model is used in the web app.

```
model = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(max_iter=1000, random_state=42)),
])
model.fit(X_train, y_train)
joblib.dump(model, "model.pkl")
```

12. Model Evaluation

Metric	Value
Training accuracy	74.1%
Testing accuracy	82.1%
5-fold cross-validation accuracy	74.5% (± 4.1%)
ROC-AUC (test set)	0.880

The five cross-validation fold accuracies were 72.6%, 75.0%, 67.9%, 77.4%, 79.8%. The classification report for the 84 test rows is shown below.

Class	Precision	Recall	F1-score	Support
Poor (0)	0.81	0.86	0.84	44
Good (1)	0.84	0.78	0.81	40
Accuracy			0.82	84
Macro average	0.82	0.82	0.82	84

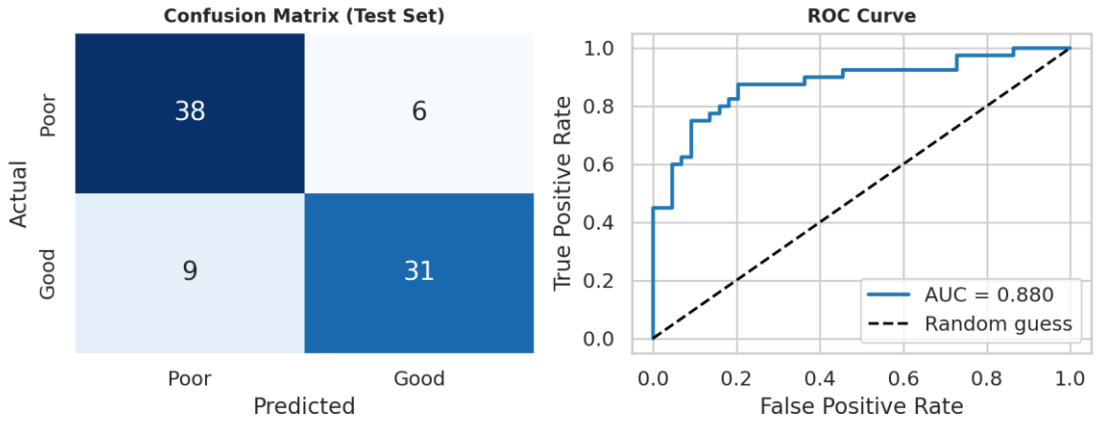


Figure 4: Confusion matrix and ROC curve on the test set

The confusion matrix shows 38 true negatives, 6 false positives, 9 false negatives and 31 true positives. Of the 84 test records, 69 were classified correctly. The model is slightly better at recognising Poor sleep (recall 0.86) than Good sleep (recall 0.78). A ROC-AUC of 0.88 means the model ranks a randomly chosen Good-sleep person above a randomly chosen Poor-sleep person about 88% of the time.

13. Feature Importance and Interpretation

The coefficients of a Logistic Regression model show how each feature changes the odds of good sleep. Because the features were standardised, the coefficients are directly comparable. The odds multiplier is exp(coefficient) and gives the change in the odds of good sleep for an increase of one standard deviation in that feature.

Feature	Coefficient	Odds multiplier (+1 SD)	Effect on sleep
exercise_hours	+1.141	3.13	Strongest positive effect
screen_time	-0.957	0.38	Strong negative effect
caffeine_intake	-0.679	0.51	Moderate negative effect
age	-0.060	0.94	Very small effect
gender_Male	-0.152	-	Small effect (vs Female)
gender_Other	-0.245	-	Small effect (vs Female)

Exercise is the strongest protective factor, followed (in the negative direction) by screen time and caffeine. This agrees with the rule used to create the data and with the EDA. The gender coefficients are small, and the dataset has only 12 records in the other group, so they should not be over-interpreted.

14. Overfitting and Underfitting Analysis

Overfitting is indicated when training accuracy is much higher than test accuracy, and underfitting when both are low. Here the training accuracy is 74.1%, the test accuracy is 82.1% and the cross-validated accuracy is 74.5%. The test score is higher than the training score by 8.0 points, which is the opposite of overfitting. This is caused by the small test set of 84 rows, where a few easy cases change the score noticeably. The cross-validated accuracy (74.5%) is almost the same as the training accuracy (74.1%), which is the more reliable evidence that the model is stable and generalises well.

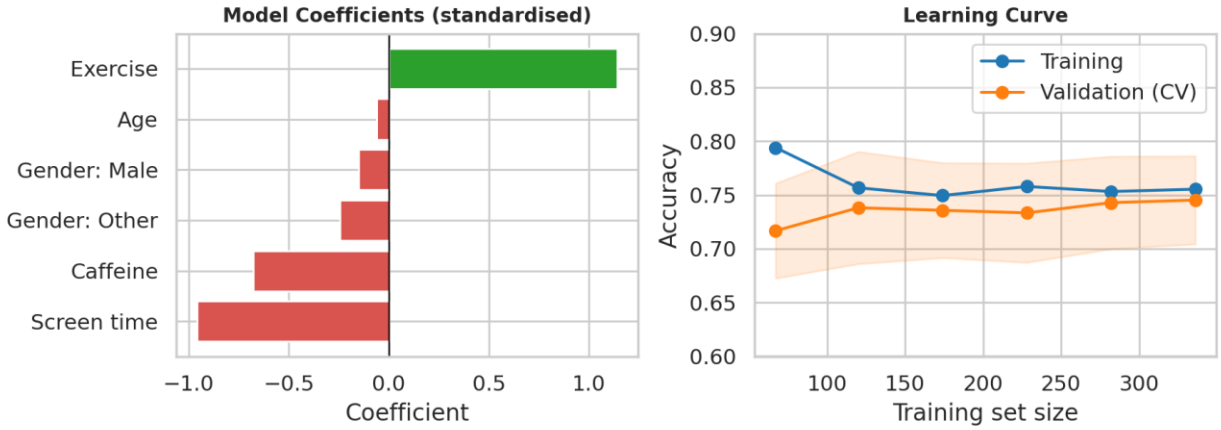


Figure 5: Model coefficients (left) and learning curve (right)

The learning curve shows the training and validation accuracy converging at about 75% as more data is used. This means the model has reached the limit set by the **noise in the data** (some people with healthy habits still sleep poorly and the reverse), not a limit caused by memorising the training set. Moderate accuracy with a small gap between training and validation is therefore expected for this dataset, and adding model complexity would be unlikely to help.

15. System Architecture

The system is organised as two connected pipelines. The model-development pipeline loads and cleans the CSV file, performs EDA, encodes the data, splits it and trains and evaluates the model. The trained pipeline (scaler and classifier) is saved as model.pkl. The deployment pipeline is the Streamlit application, which loads model.pkl, takes the user's habits as input, predicts the probability of good sleep and generates the recommendations.

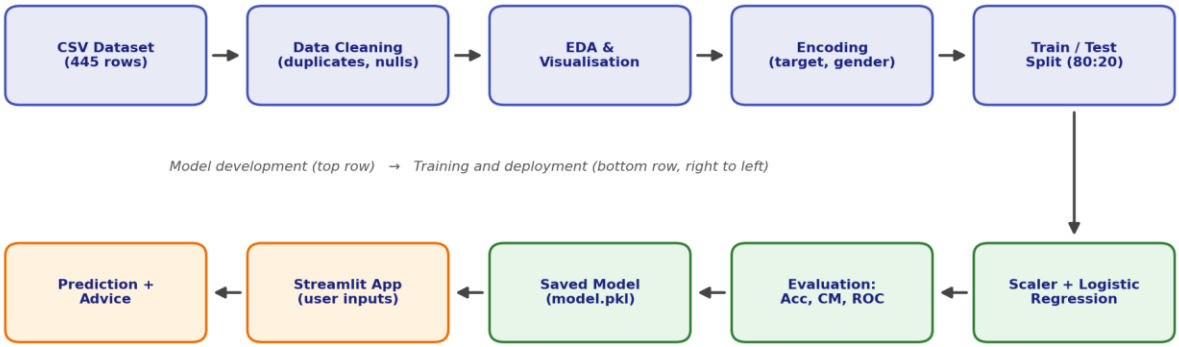


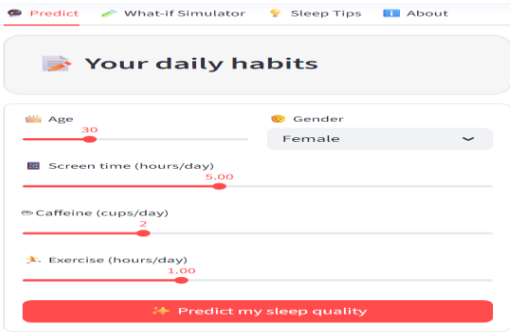
Figure 6: System architecture of Sleep Quality Predictor

16. Streamlit User Interface

The Streamlit application provides an attractive and easy-to-use front end. It has a gradient header and four tabs: **Predict**, **What-if Simulator**, **Sleep Tips** and **About**. In the Predict tab, the user sets age, gender, screen time, caffeine intake and exercise hours using sliders and a selection box, and presses the Predict button. The application then displays:

- a coloured **badge** (Good or Poor sleep likely) and a **semicircle gauge** showing the sleep score out of 100, coloured green at 65 and above, amber from 50 to 64 and red below 50;
- the **probability** of good and poor sleep and a short summary message;
- three **habit tiles** for screen time, caffeine and exercise, each with a status chip (for example Healthy, Moderate or High);
- a **factor chart** that shows which habits push the result towards Good or Poor sleep, using the model coefficients multiplied by the standardised inputs;
- **personalised advice cards** (see Section 17).

The What-if Simulator lets the user change screen time, caffeine and exercise and immediately see the new sleep score and the change in points, which makes the benefit of better habits easy to understand. The last prediction is kept in Streamlit session state, so the user can move between tabs without losing it. The model is loaded once with st.cache_resource.



Input From and Habits



Prediction Gauge

Figure 7: Stream lit application interface

17. Personalised Advice Engine

After the prediction, the application recommends advice based on the prediction and on the user's own habits. The advice is generated by simple, transparent rules, so every recommendation can be explained:

Habit	Condition	Advice given
Screen time	Above 6 h	Cut screen time; aim for under 4-5 h and avoid screens 60 minutes before bed
	4 to 6 h	Trim slightly; set a screen curfew an hour before bed
	4 h or less	Healthy; keep screens out of the bedroom
Caffeine	4 or more cups	Reduce to 2 cups or fewer; last cup before 2 PM
	3 cups	Keep caffeine to mornings and early afternoon
	2 cups or fewer	Under control; keep it that way
Exercise	Below 0.5 h	Start with a 20-30 minute daily walk and build up to about 1 hour
	0.5 to 1 h	Add a little more; aim for about 1 hour a day
	1 h or more	Great habit; regular activity deepens sleep
Routine	Prediction is Poor / Good	Build a fixed sleep schedule and a calm bedroom / protect the current routine
Age	45 or older	Limit late fluids and heavy dinners; get morning sunlight

Each tip is shown in a card with a coloured edge (green for good, amber for a warning, red for needing action). The application also states clearly that it is a learning project and not medical advice, and encourages users with long-term sleep problems to consult a doctor.

18. Complete System Workflow

The complete workflow of the system is summarised below.

1. Load sleep_quality_data.csv with pandas and inspect it (shape, info, describe, nulls, duplicates). **2.** Remove duplicates, drop id and fill the missing values. **3.** Carry out EDA with histograms, bar charts, boxplots and the heatmap. **4.** Encode the target and one-hot encode gender. **5.** Split into training and testing sets (80:20, stratified). **6.** Train the scaler and Logistic Regression pipeline. **7.** Evaluate with accuracy, confusion matrix, classification report, ROC-AUC and cross-validation. **8.** Save the model as model.pkl and the feature list as feature_columns.pkl. **9.** The Streamlit app loads the model, builds a one-row input in the same feature order, predicts the probability of good sleep and displays the score, factor chart and advice.

User Inputs → Feature Row → Scaler → Logistic Regression → P(Good) → Score / Label → Advice Rules → Streamlit UI

19. Testing and Validation

The project was tested for data handling, model behaviour and the user interface. The model tests below used sample people with different habits; the probabilities are the model's actual predicted chance of good sleep.

Test ID	Test Input / Action	Expected Result
TC01	Cleaning: load the raw CSV and remove duplicates	445 rows reduced to 420; no duplicate rows remain
TC02	Cleaning: fill missing values	No null values remain in any column
TC03	Healthy habits: age 25, Female, screen 2 h, 1 cup, exercise 1.5 h	Good sleep; the model gives 95.8% chance of Good
TC04	Unhealthy habits: age 35, Male, screen 9.5 h, 6 cups, exercise 0.1 h	Poor sleep; the model gives 0.6% chance of Good

Test ID	Test Input / Action	Expected Result
TC05	Borderline habits: age 30, Female, screen 5 h, 2 cups, exercise 1 h	Good sleep, amber score; 63.2% chance of Good
TC06	What-if: screen 8 h, 4 cups, exercise 0.25 h changed to 4 h, 2 cups, 1 h	Score rises from 3.8% to 65.5%
TC07	Run the app without model.pkl	A clear error message asks the user to run the notebook first
TC08	Press Predict in the app	Badge, gauge, habit tiles, factor chart and advice cards appear
TC09	Move to another tab and back	The last prediction is still displayed (session state)
TC10	Use the sliders at their limits	Inputs stay within the allowed ranges (age 18-60, screen 0-12 h, caffeine 0-8, exercise 0-3 h)

20. Results and Discussion

The project demonstrates a complete and understandable machine learning workflow. The preprocessing pipeline turned a messy dataset (445 rows with 25 duplicates and 64 missing values) into 420 clean records. The EDA clearly showed the expected relationships between habits and sleep, and the model captured them: exercise raises the odds of good sleep, while screen time and caffeine lower them.

The Logistic Regression model achieved **82.1% test accuracy**, **74.5% cross-validated accuracy** and an **AUC of 0.88**. The precision and recall values for both classes are balanced (between 0.78 and 0.86), so the model does not favour one class strongly. The small gap between training and cross-validated accuracy shows that the model is not overfitting; the remaining error mostly comes from the random noise built into the synthetic data.

The Streamlit application makes the results practical. Instead of showing only a label, it gives a probability, explains the contributing factors and recommends concrete actions, and the what-if simulator shows the likely benefit of changing a habit. Because the dataset is synthetic, the accuracy figures should be read as a demonstration of the method, not as a claim about real-world sleep.

21. Advantages

- Simple, fast and easy to explain: Logistic Regression gives interpretable coefficients and probabilities.
- Complete workflow from raw, messy data to a deployed application in one project.
- A scikit-learn Pipeline prevents data leakage and keeps training and prediction consistent.
- The Streamlit interface needs no front-end knowledge and the app runs locally without any external API.
- Advice is rule-based and transparent, so users can see why each recommendation is given.
- The what-if simulator motivates users by showing how small changes can improve their score.

22. Limitations

- **Synthetic data:** the dataset was generated by a script, so the model reflects the generating rule and not real human sleep.
- **Limited features:** sleep quality also depends on stress, diet, sleep duration, health conditions and the sleep environment, which are not included.
- **Binary target:** only Good and Poor are predicted; there is no scale between them.
- **Small test set:** with 84 test rows, the test accuracy can change noticeably from one split to another, so cross-validation is more reliable.

- **Imputation before the split:** missing values were filled using the whole dataset before splitting; strictly, the medians should be computed on the training set only. The effect is very small here.
- **Linear model:** Logistic Regression assumes a linear relationship between the standardised features and the log-odds of good sleep.
- **Not medical advice:** the application is for education and must not be used for diagnosis.

23. Future Scope

- Collect **real data** through surveys or wearable devices, and add features such as sleep duration, stress level, diet and bedtime.
- Compare other models (Random Forest, Gradient Boosting, SVM) with Logistic Regression and tune the hyperparameters.
- Predict a sleep score or several sleep categories instead of only Good and Poor.
- Add user accounts and a database so users can track their habits and sleep score over time.
- Deploy the application to the cloud (for example Streamlit Community Cloud) and add monitoring.
- Use a generative AI model to produce more natural and personalised coaching messages.
- Add multilingual support and a mobile-friendly interface.

24. Conclusion

Sleep Quality Predictor demonstrates the full machine learning workflow on a practical, relatable problem. A custom dataset with missing values and duplicates was cleaned with pandas, analysed with histograms, bar charts, boxplots and a correlation heatmap, encoded and split, and used to train a Logistic Regression pipeline. The model reached 82.1% test accuracy, 74.5% cross-validated accuracy and an AUC of 0.88, with no sign of overfitting.

The trained model was deployed in an attractive Streamlit application that predicts sleep quality, explains the influencing factors and gives personalised advice. The project meets the objectives of the assignment and provides a solid foundation for future work with real data and richer models.

25. References

The following official documentation was used while developing and maintaining the project. Exact access dates can be added according to the citation style required by the institution.

Technology	Reference
Python	Official Document
pandas	Official Document
NumPy	Official Document
scikit-learn	Official Document
Matplotlib / Seaborn	Official Document
Streamlit	Official Document